Technical introduction to PSOC™ Edge E84 features

Chapter 2: Audio

About this document

Scope and purpose

This document is the training manual for the technical introduction to PSOC™ Edge E84 features. This manual helps in getting started with different applications leveraging key features for PSOC™ Edge, using ModusToolbox™ software.

The training is divided into multiple chapters covering topics like audio, graphics, machine learning, among others.

Chapter 2 discusses PSOC™ Edge's audio features.

Intended audience

The document is intended for design engineers, technicians, and developers of electronic systems.

Table of contents

About this document **1**
Table of contents **2**
1 Introduction **3**
2 Prerequisites **4**
2.1 Tools for audio labs 4
3 Code example: PDM to I2S **5**
3.1 Objective 5
3.2 Description 5
3.3 Audio data flow 5
3.4 Hardware overview 6
3.5 Project creation 6
3.6 Output 11
3.7 Modify application 11
3.8 Conclusion 12
4 Code example: Mains-powered local voice **13**
4.1 Objective 13
4.2 Description 13
4.3 Audio data flow 13
4.4 Hardware overview 15
4.5 Project creation 15
4.6 Output 18
4.7 Modify application 19
5 Appendix A: Creating a PSOC™ Edge application in ModusToolbox™ **26**
6 Appendix B: KIT_PSE84_EVAL details **29**
Revision history **30**
Disclaimer **31**

1 Introduction

This manual provides detailed instructions to create, configure, build, and run application code examples on the PSOC™ Edge E84 MCU.

2 Prerequisites

See the **Prerequisites** section in the **Chapter 1 training manual** for a complete list of prerequisites and development tools.

2.1 Tools for audio labs

Note that audio labs utilize the following additional tools:

- LLVM for Arm® v19.1.5 or later if recommended by ModusToolbox™
  - The code example used in Code example: Mains-powered local voice requires the LLVM for Arm® or the Arm® compilers. The instructions in this manual are for using LLVM.
  - Set the environment variable **CY_COMPILER_LLVM_ARM_DIR** to point to the installation directory if you intend to use LLVM for other projects. Otherwise, this definition can be added to the project Makefile.
- DEEPCRAFT™ Voice Assistant
  - You will need to create an account for this tool. This tool is used with the Code example: Mains-powered local voice. It is not required to build the default example, but it is utilized to modify the wake word and commands.
- DEEPCRAFT™ Audio Enhancement Tech Pack v1.0.0.90 or later
  - This tool is not required to build and perform the modifications suggested in this manual; however, this guide provides an introduction and mentions some of its features.

3 Code example: PDM to I2S

3.1 Objective

This code example demonstrates how to route pulse-density modulation (PDM) audio data to the inter-IC sound (I2S) interface in PSOC™ Edge MCU. For this code example the PDM/PCM and I2S hardware blocks are configured to be used by the Arm® Cortex®-M33 CPU.

3.2 Description

This code example shows how to record a short audio sample from a microphone and then play it on a speaker or headphone. The example uses the PDM/PCM block to interface with a digital microphone. All recorded data is stored in the internal SRAM. Once the recording completes, the I2S block starts sending data to an external audio codec TLV320DAC3100. Press the user button on the kit to record an audio sample. When the button is released, it plays back the recorded audio.

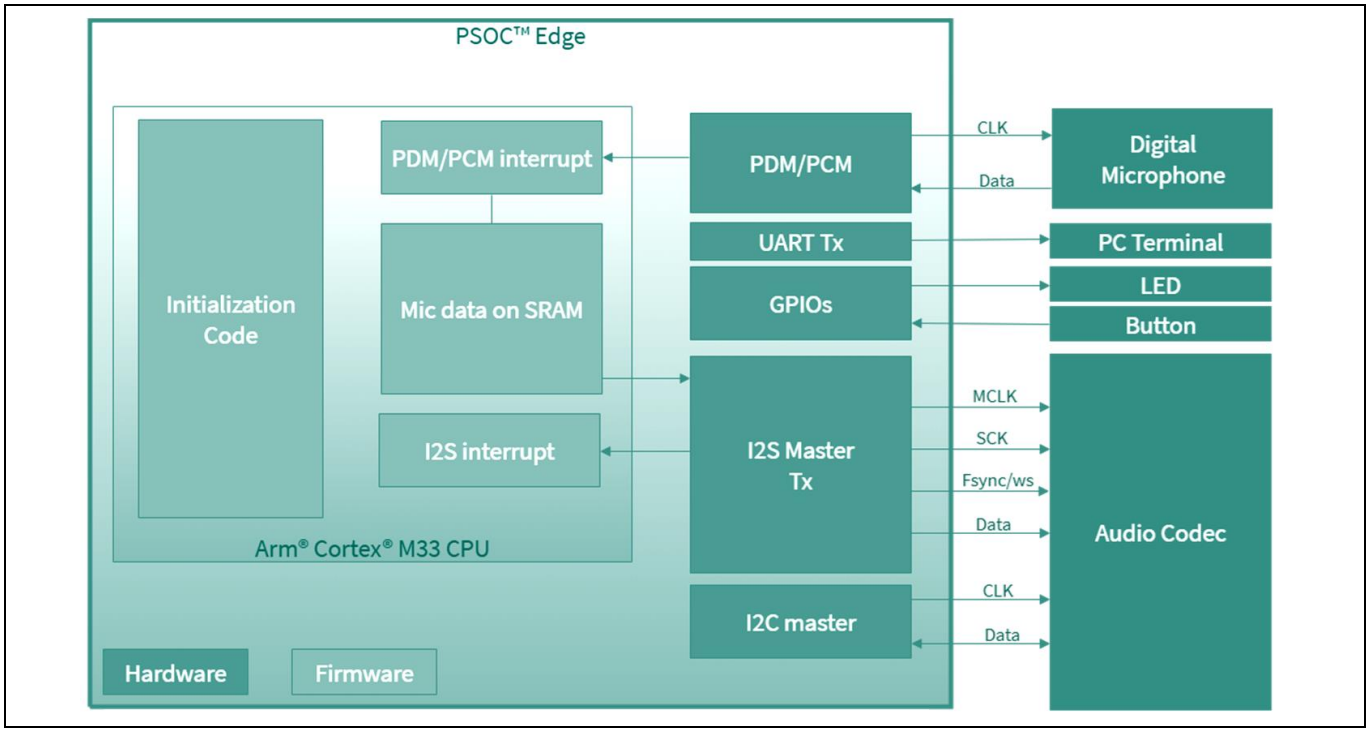


Figure 1 Block diagram for PDM to I2S code example

3.3 Audio data flow

Audio data is sampled by PDM mics on EVK in PDM format. PDM is best summarized as "oversampled 1-bit audio" as it is nothing more than a high-sampling rate, single-bit digital signal. Most current digital audio systems use multi-bit PCM to represent the signal. The PDM/PCM block converts PDM data to PCM data. PCM's advantage is that it is easy to manipulate. This enables signal processing operations on the audio stream, such as mixing, filtering, and equalization.

The converted PCM audio data is stored in SRAM. Audio data is played over the speaker on the EVK board. The PCM data is sent to the onboard hardware codec (TLV320DAC3100) over I2S. The speaker is connected to the TLV320DAC3100.

The high-level data flow of audio data is shown in Figure 2.

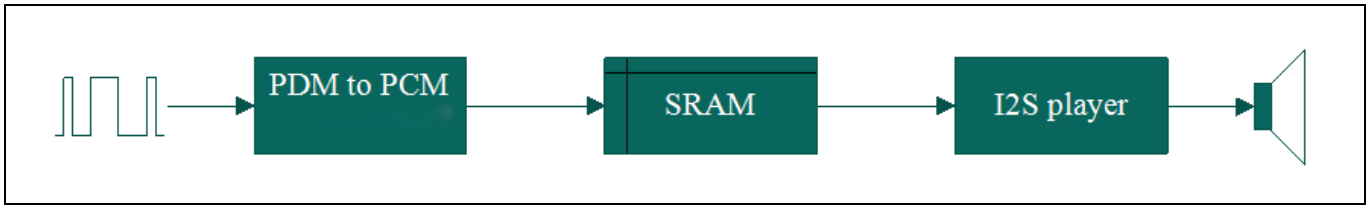
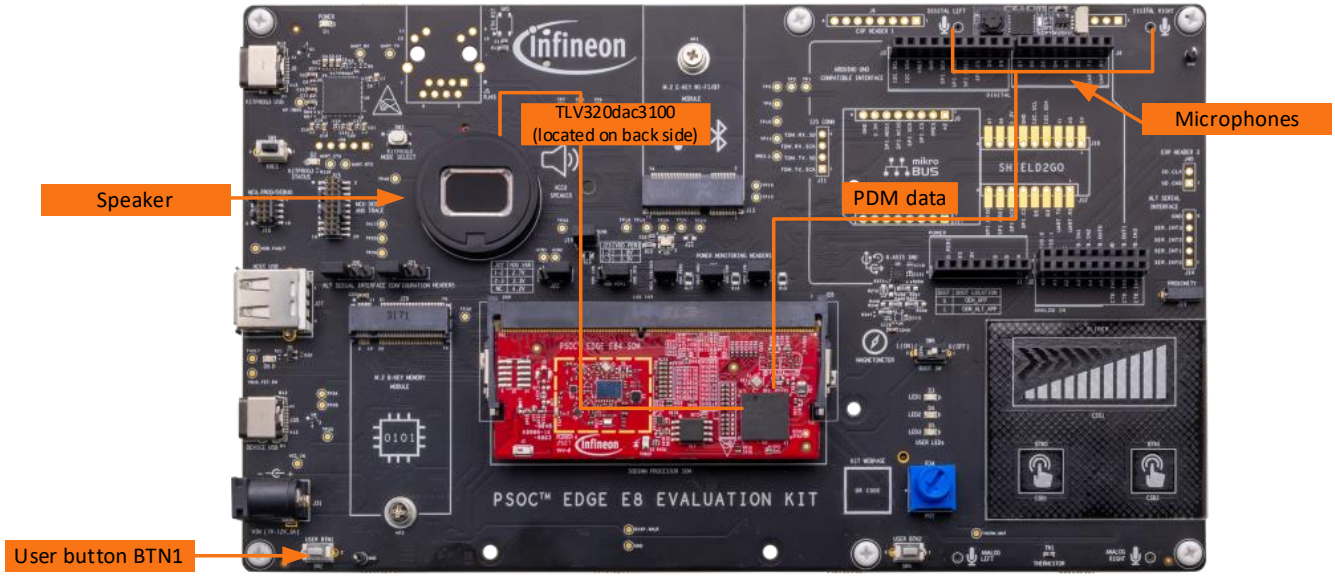


Figure 2 Data flow of PDM to I2S example

3.4 Hardware overview

Note: This code example requires BOOT SW in ON position.

See Appendix B: KIT_PSE84_EVAL details for more details.



Note: the signals shown are for illustration purposes, not exact routing

Figure 3 Hardware diagram for PDM to I2S code example

3.5 Project creation

1. Follow steps 1-4 in Appendix A: Creating a PSOC™ Edge application in ModusToolbox™ to create a new application.
2. When creating the application, select the **PSOC™ Edge PDM to I2S** application under the **Peripherals** section.

Note: To prevent issues with Windows path length, it's recommended to rename the project if your worskpace path is long.

Code example: PDM to I2S

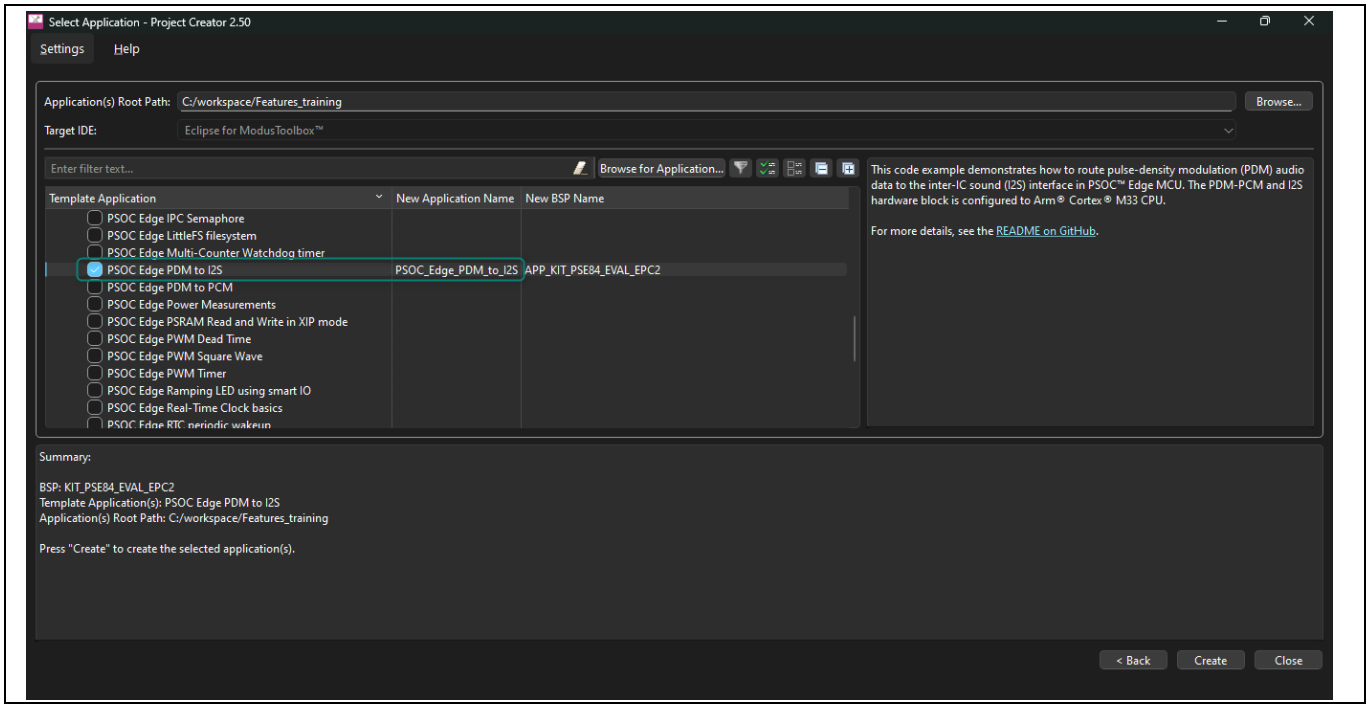


Figure 4 Creating PDM to I2S code example

3. **Note**: PDM/PCM and I2S are configured by the **Device Configurator**.
   No changes will be implemented at this point, but you can try opening the **Device Configurator** to observe the configuration.

   a) Clock configuration

   – Both PDM/PCM and I2S use the same HF clock that is HF7. Due to fractional clock frequency requirements, one PLL is dedicated to audio clocks.

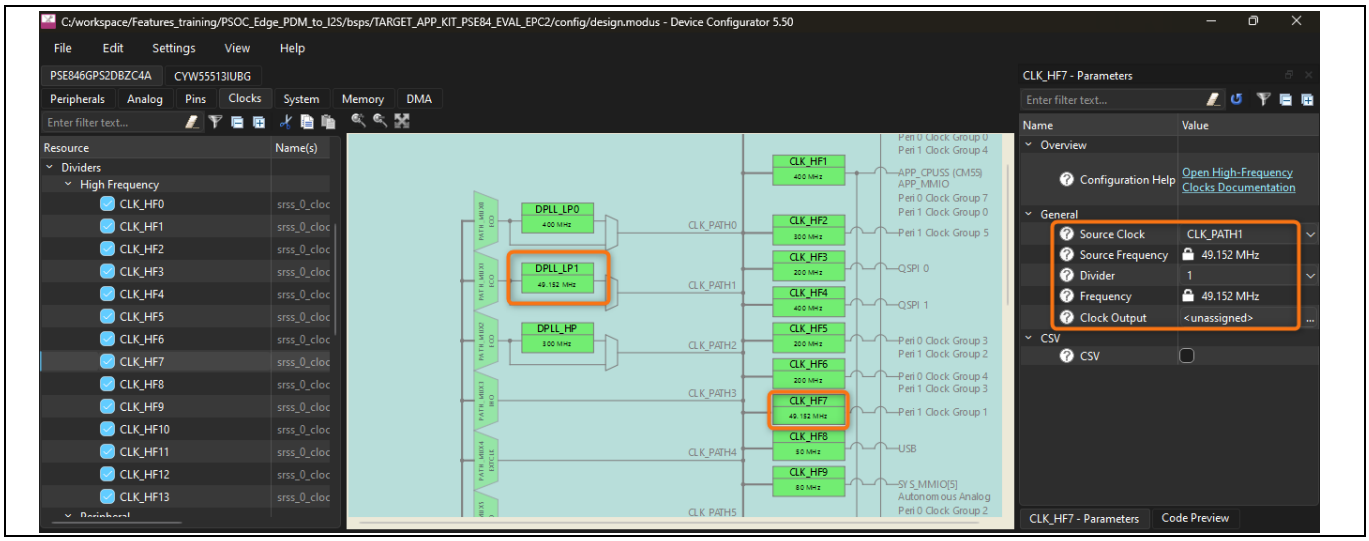


Figure 5 HF7 clock for PDM/PCM and I2S

– The PDM clock is configured as follows.

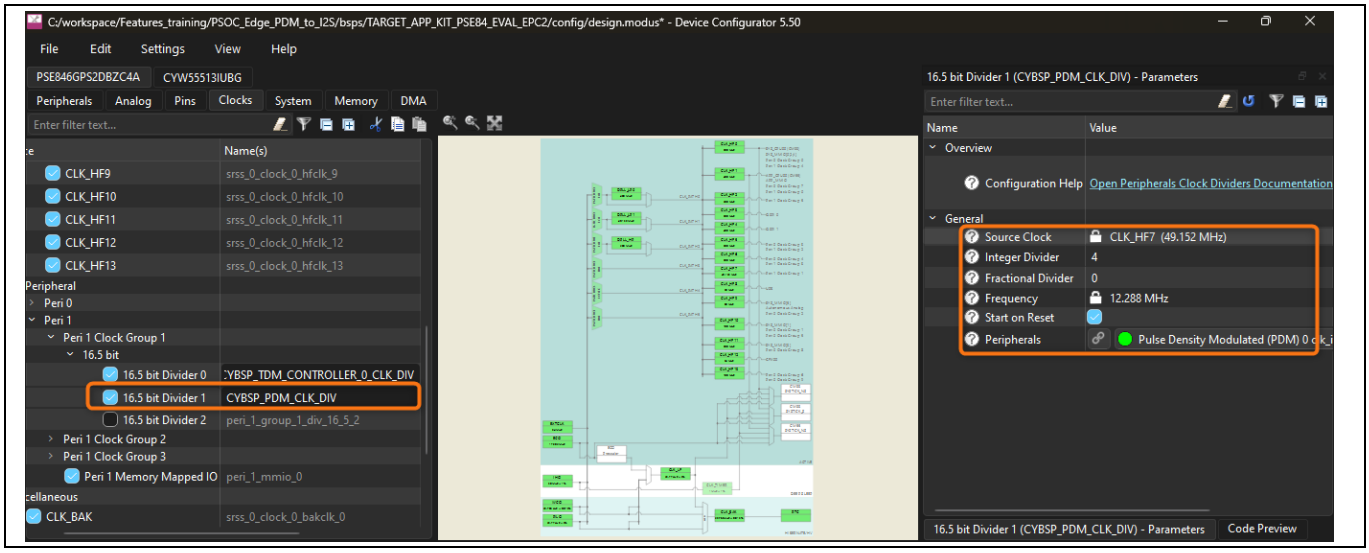


Figure 6 Peripheral clock configuration for PDM

– The clock path will be:
DPLL_PLL → HF7 → 16.5 fractional divider → PDM/PCM interface clock (clk_if_srss)

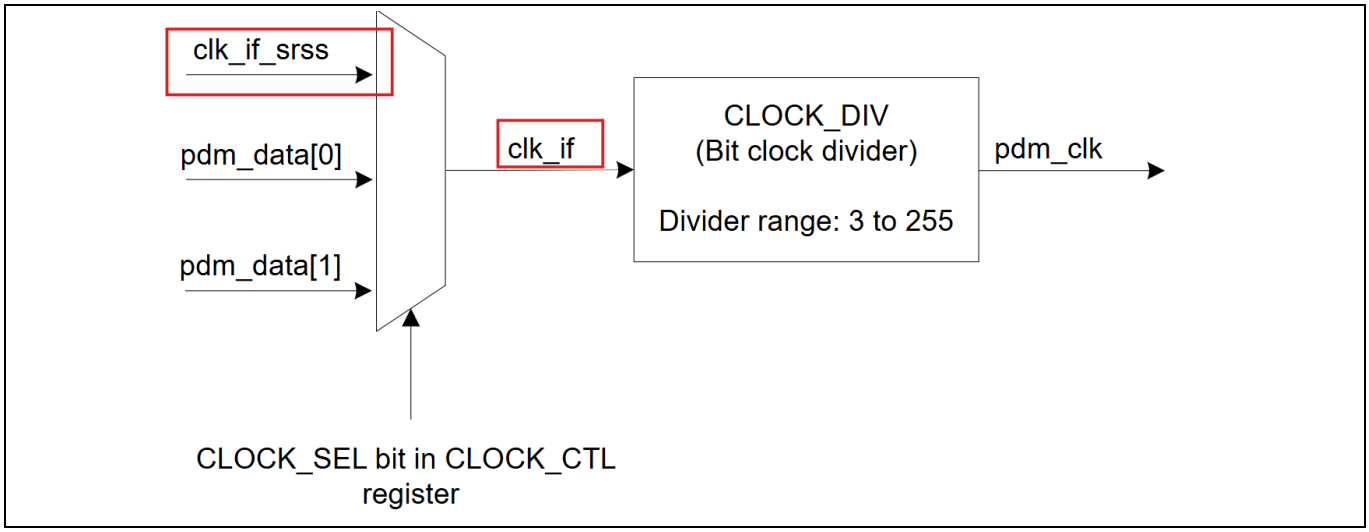


Figure 7 Clock generation for PDM

– The I2S clock configuration is as shown in Figure 8.

Code example: PDM to I2S

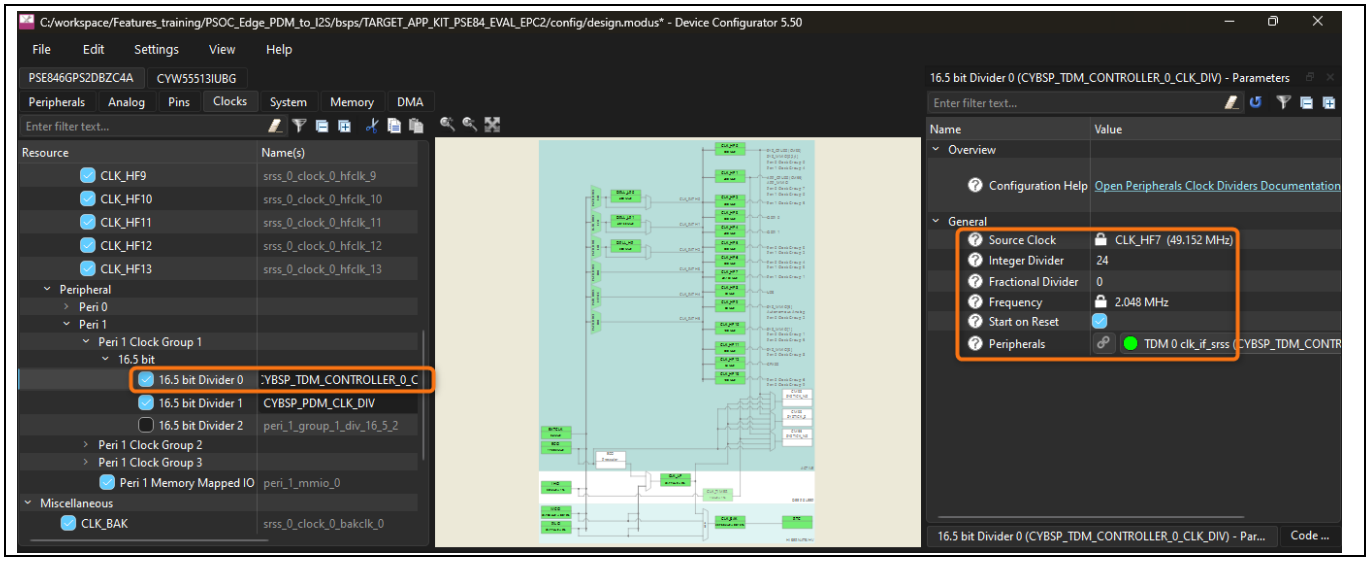


Figure 8 Peripheral clock configuration for I2S

- The clock path is:
  DPLL_PLL → HF7 → 16.5 fractional divider → I2S interface clock (clk_if_srss[0:3])

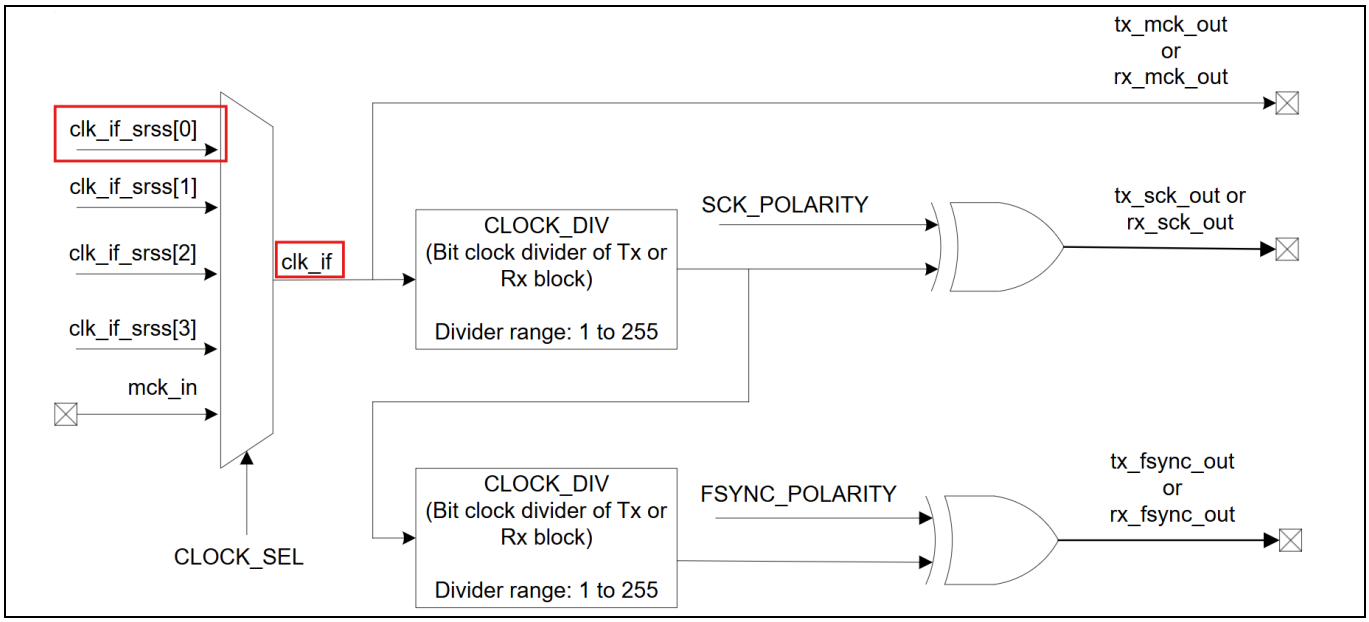


Figure 9 Clock generation for I2S

b) In the **Peripherals** tab, observe the PDM/PCM configuration: **Digital** > **Pulse Density Modulated (PDM) 0** as shown in Figure 10.

Code example: PDM to I2S

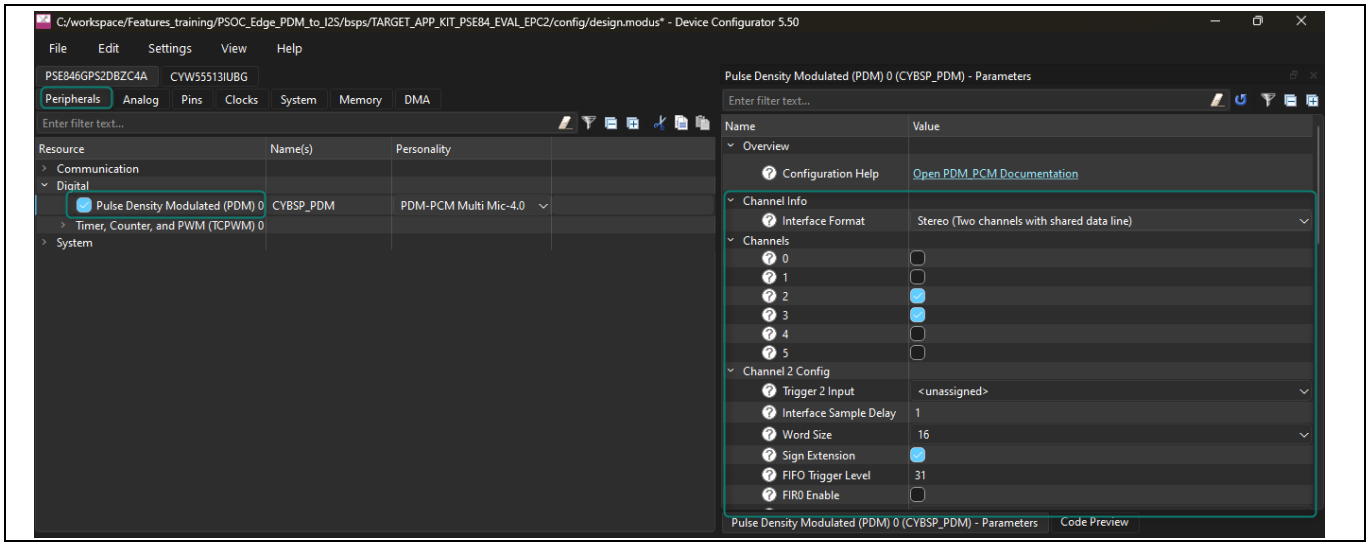


Figure 10 PDM configuration in Device Configurator

c) Observe the I2S configuration in **Communication** > **Time Division Multiplexing (TDM)** > **TDM 0**

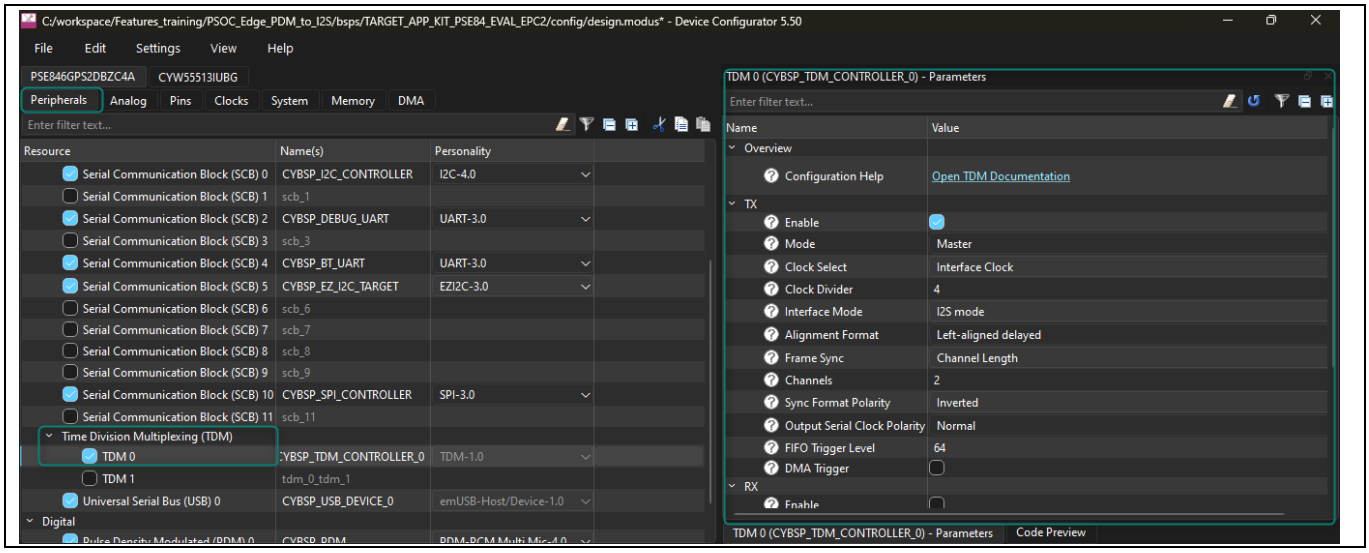


Figure 11 I2S configuration in Device Configurator

4. **Note**: No modifications are necessary to build the default code; but feel free to observe the source code.
   a) The application is implemented using the non-secure CM33 project, so the main file is located at:
   <PROJ_DIR>/PSOC_Edge_PDM_to_I2S.proj_cm33_ns/main.c
   b) The source code to initialize and handle PDM data from microphones is implemented at:
   <PROJ_DIR>/PSOC_Edge_PDM_to_I2S.proj_cm33_ns/source/app_pdm_pcm/
   c) The source code to initialize and handle output I2S data to speakers is at:
   <PROJ_DIR>/PSOC_Edge_PDM_to_I2S.proj_cm33_ns/source/app_i2s/
5. Build and program the application.

3.6 Output

After successful programming, press and hold the user button BTN1 on the PSOC™ Edge E84 kit and record audio. Release the button to play back the audio on the kit speaker.

Optionally, open a terminal to observe the application running.



Figure 12 PDM to I2S example output

3.7 Modify application

There are many configurable parameters in this code example: the PDM/PCM, the I2S, and the TLV320DAC3100 hardware codec. Let's try to change the playback volume. This provides a foundation for further experimentation and optimization of the system.

Hints for advanced modification

- Look for a codec API to modify volume
- The codec is implemented in mtb_tlv320dac3100

Solution

1. Open the mtb_tlv320dac3100 API reference available in GitHub at: https://github.com/Infineon/audio-codec-tlv320dac3100/blob/master/API_reference.md
2. Observe the code snippet to adjust the volume of the speaker and headphone. However, notice that our current setup is using the speaker and not the headphone. https://github.com/Infineon/audio-codec-tlv320dac3100/blob/master/API_reference.md#snippet-2-volume-control-of-speaker-and-headphone
3. Open *proj_cm33_ns/main.c* and modify the lines highlighted in the code listing.

Code listing 1 PDM to I2S modification – Increase speaker volume

```
/* Initialize the PDM-PCM block */
app_pdm_pcm_init();

/* Deactivates TLV320DAC3100 audio codec. */
mtb_tlv320dac3100_deactivate();

/* Updates the volume of both the left and right channels of the speaker output. */
```

```
mtb_tlv320dac3100_adjust_speaker_output_volume(0xFE);

/* Activates TLV320DAC3100 audio codec. */
mtb_tlv320dac3100_activate();

/* Enable CM55. */
```

Note: mtb_tlv320dac3100_adjust_speaker_output_volume takes an 8-bit value as a parameter which represents the volume in 0.5dB increments.

4. Save the changes to the file, rebuild, and reprogram the updated code.
5. Test the change by recording and playing back some audio.
   If the volume seems too high or too low, you can readjust the value until you achieve the desired volume.

3.8 Conclusion

We successfully built and modified the PSOC™ Edge E84 PDM to I2S code example using ModusToolbox™ and figured out how to use the PDM/PCM and I2S blocks in the PSOC™ Edge E84 MCU.

This example is a foundation for other audio code examples since it shows how to receive data using PDM microphones and how to transfer to the speaker using I2S.

4 Code example: Mains-powered local voice

4.1 Objective

This code example demonstrates how to implement the mains-powered local voice use case, using the audio front end (AFE) components and Infineon's DEEPCRAFT™ solutions for wake word detection and automatic speech recognition (ASR). By understanding how the AFE Components work together, and customizing them for your specific application, you can optimize the performance of the system and enhance the user experience.

"Mains-powered" simply means that the use-case for this application is to be always on and listening and therefore cannot be powered by a battery.

Note that the ITSI inferencing library solution and corresponding code examples are available only for the Arm® and LLVM compilers. These instructions use the LLVM compiler.
Also note that the audio-voice-core library and voice assistant inferencing library included in this code example have a limited operation of about 15 minutes and 30 minutes respectively. For an unlimited licence, please contact your Infineon support.

4.2 Description

The code example is designed to detect voice commands and activate wake word detection using PSOC™ Edge MCUs. It demonstrates how to use an audio pipeline on the Arm® Cortex®-M55 core, starting from audio capture, audio data processing using Infineon's DEEPCRAFT™ Audio Enhancement (AE), audio data inferencing with Infineon's DEEPCRAFT™ Voice Assistant (VA), post-processing of inferred results, and performing actions based on the results. Both VA and AE components (noise suppression and echo suppression) utilize Ethos™-U55.

The code example leverages the display to show a music player with 2D graphics and uses "OK Infineon" as the wake word and supports several commands/intents.
The following block diagram shows a high-level overview of the audio pipeline:

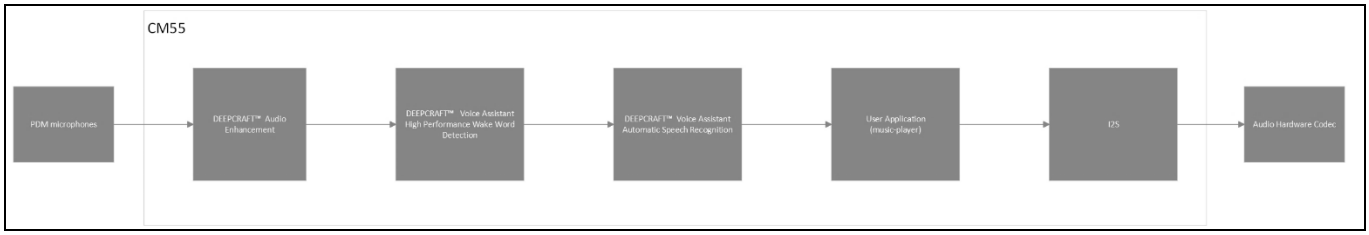


Figure 13 Mains powered local voice audio pipeline

4.3 Audio data flow

The entire audio pipeline runs on Arm® Cortex® M55 (CM55). PDM mics are controlled by the CM55 application.

The audio frames from the PDM microphone are fed into audio enhancement (AE). The processed audio frames are then sent to the inferencing engine for wake word and command detection. Based on the inferred data, the required action is taken, and the pre-recorded audio data is played over I2S on the onboard speakers.

The audio front end (AFE) middleware has multiple software components:

- High-pass filter
- Analysis
- Acoustic echo cancellation (AEC)

- Beamforming
- Dereverberation
- Echo suppression
- Noise suppression
- Synthesis

The audio data flows through each component of AFE depending on whether it is enabled or not. The microphone data is then filtered using a high-pass filter to remove any low-frequency noise or interference. The AEC block uses reference/playback audio data to cancel out any acoustic echoes. The noise suppression component is used to reduce background noise and improve the clarity of the audio signal. Overall, the AFE components work together to enhance the quality of the audio signal and improve the inferencing performance.

The noise suppression and echo suppression components are ML-based and are executed on Ethos™-U55. The processed audio data is then sent to the voice assistant for wake word detection and automatic speech recognition.

The code flow of this application is shown in Figure 14 for the “OK Infineon” wake word and “Play Music” command.

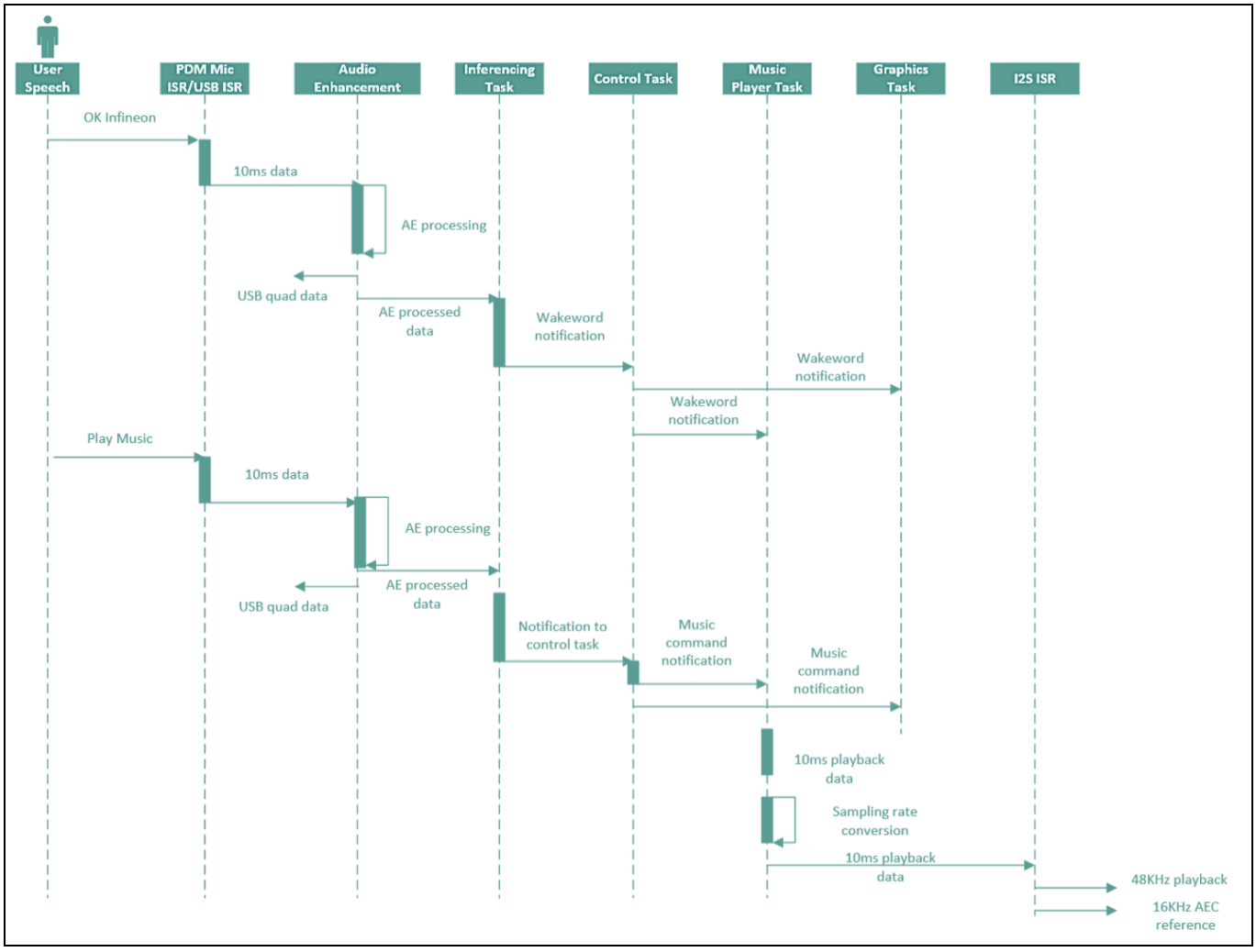
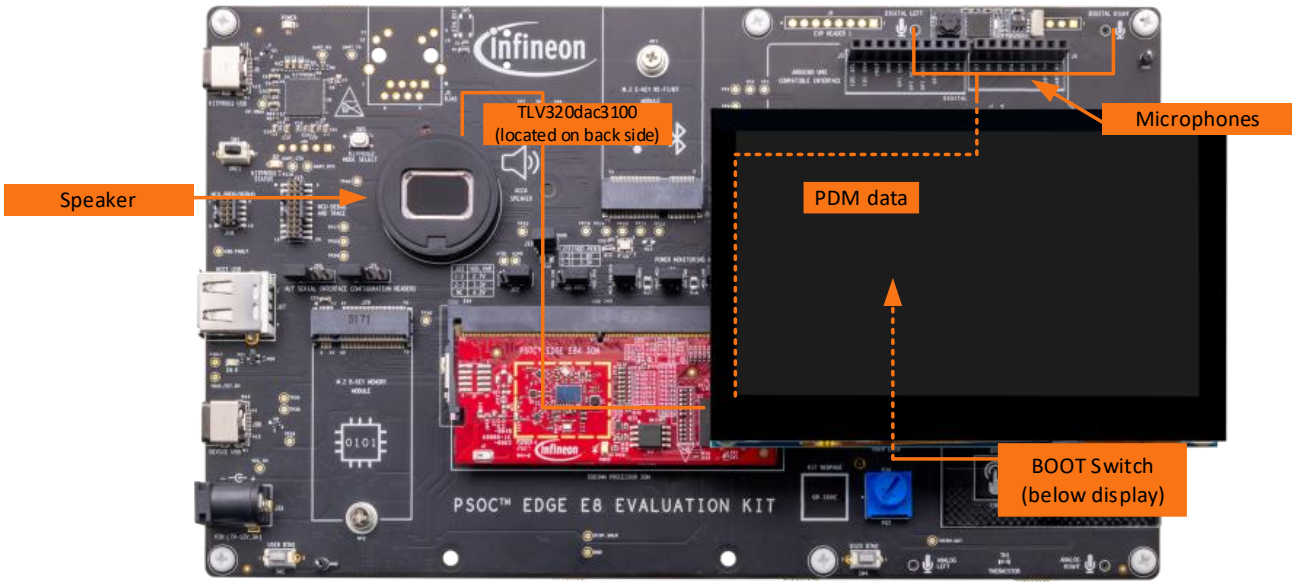


Figure 14 Code flow for mains powered local voice application

4.4 Hardware overview

Note: This example requires BOOT SW in OFF position, and it uses the display to show a music player.

See Appendix B: KIT_PSE84_EVAL details for more details.



Note: the signals shown are for illustration purposes, not exact routing

Figure 15 Hardware diagram for mains-powered local voice application

4.5 Project creation

1. Follow steps 1-4 in Appendix A: Creating a PSOC™ Edge application in ModusToolbox™ to create a new application.
2. When creating the application, select the **PSOC™ Edge Mains Powered Local Voice** application under the **Audio** section.

Note: To prevent issues with Windows path length, it's recommended to rename the project if your worskpace path is long.

Code example: Mains-powered local voice

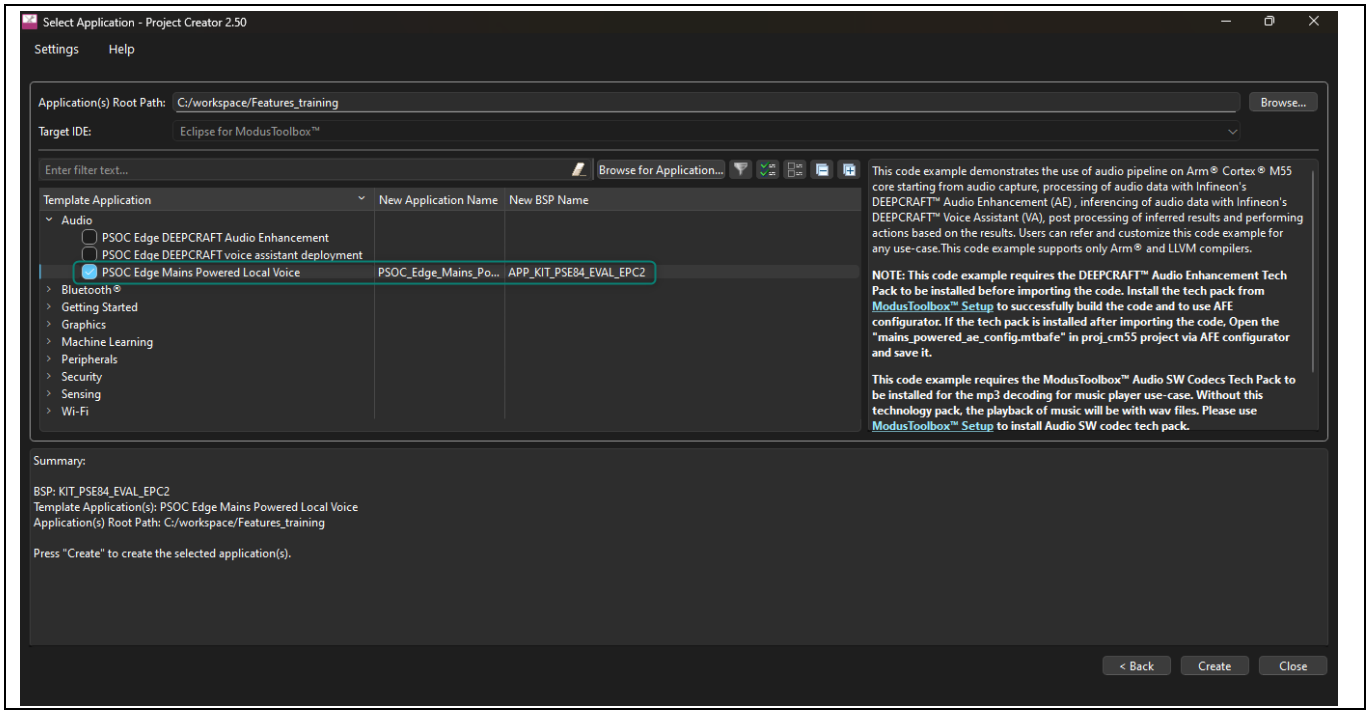


Figure 16 Creating the mains-powered local voice code example

3. **Note**: After the project is created, observe the project file structure.

Mains Powered Local Voice

Project Folder	SubFolders			Description
proj_cm33_ns				Non Secure CM33 project
proj_cm33s				Custom secure project executing out of RRAM
proj_cm55				
|_____ source				
| |_____ mains_powered_application				Contains the codes for Mains Powered configuration.
| | |_____ inferencing_interface				Contains wrapper for inferencing abstraction
| | | |_____			COMPONENT_VOICE_ASSISTANT	Voice Assistant wrapper and code
| | |_____ audio_enhancement				Contains wrapper and code for Audio Enhancement
|				
| |_____ modules				Modules used by Local Voice in CM55
| | |_____ audio_conversion_utils				Contains audio conversion API
| | |_____ pdm_mic_input				Contains code for PDM mic
| | |_____ push_to_talk				Contains push to talk and user button code.
| | |_____ usb_audio				Contains USB audio class code
| | |_____ audio_voice_core_lib				Placeholder folder to place MP3 decoder files once license is received.
| | |_____ sw_codec				Placeholder folder to place MP3 decoder files once license is received.
|				
| |_____ use_case				Place use-cases here.
| | |_____ COMPONENT_MUSICPLAYER				Music player use-case.
common_modules				
|_____ COMPONENT_PROFILER				Contains profiler code
|_____ COMPONENT_APP_LOGGER				Contains application logger API
templates				Contains custom linker scripts and design.modus
|_____ TARGET_KIT_PSE84_EVAL_EPC2				
| |_____ config				Custom design.modus file.
| COMPONENT_CM55				Custom Linker script for CM55 (Arm and LLVM)
|				
|_____ TARGET_KIT_PSE84_EVAL_EPC4				
| |_____ config				Custom design.modus file.
| COMPONENT_CM55				Custom Linker script for CM55 (Arm and LLVM)

Figure 17 File structure for mains-powered local voice example

4. **Note**: This code example uses the AFE Configurator tool to choose and customize the audio front end (AFE) components running on PSOC™ Edge. This tool provides a graphical user interface (GUI) that allows you to select the different AFE components, such as acoustic echo cancellation (AEC), beamforming, dereverberation, echo suppression (ES), and noise suppression.

Code example: Mains-powered local voice

After configuring the different AFE components, the AFE configurator tool generates source files, which are used directly by the application. In this way, the AFE configurator tool provides a convenient way to configure the AFE components without needing to manually edit the firmware code. It also ensures that the configuration is consistent across different projects and simplifies the process of updating the AFE configuration.

a) To open the AFE Configurator tool, select the **Audio FE Configurator** in the **Quick Panel**, under proj_cm55:
Note: DEEPCRAFT™ Audio Enhancement Tech Pack must be installed as mentioned in the section Tools for audio labs.

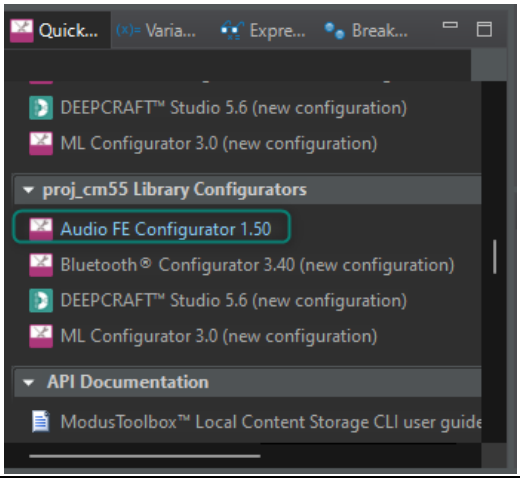


Figure 18 Opening AFE Configurator

b) Observe the different settings. For example, the figure below highlights the options to select between mono/stereo and the input source.
Note: This training is not intended to delve into AFE parameters since these topics will be covered in advanced training.

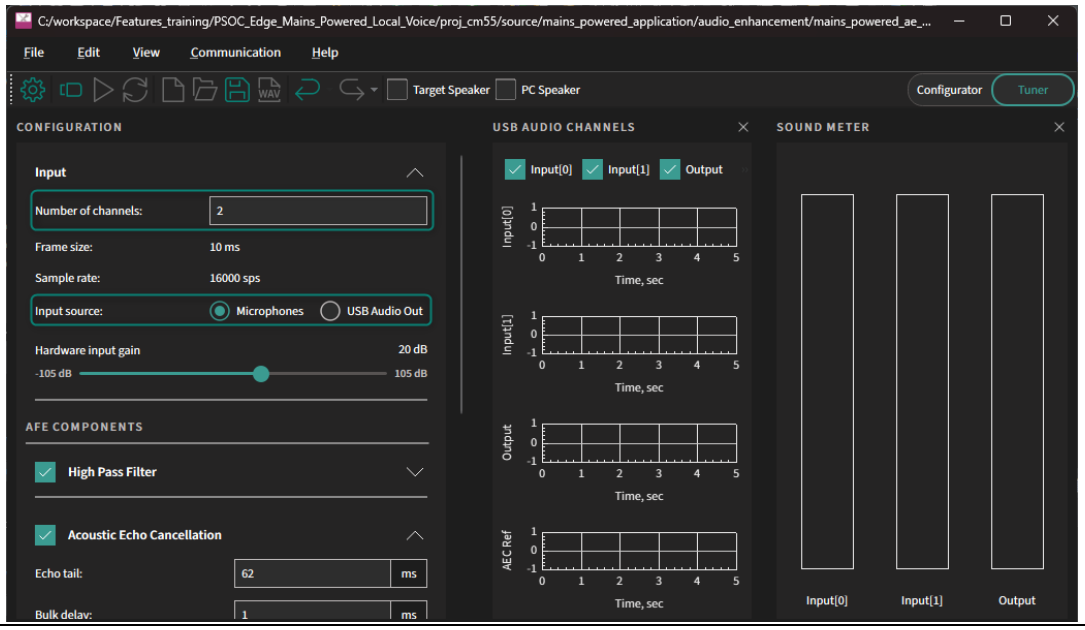


Figure 19 AFE Configurator

5. Build and program the application.

   Note: This application requires the ARM® or LLVM compiler, but it uses the latter by default.

   This guide uses LLVM as mentioned in section Tools for audio labs.
   If the environmental variable CY_COMPILER_LLVM_ARM_DIR was not defined in your system, modify common.mk as shown below.

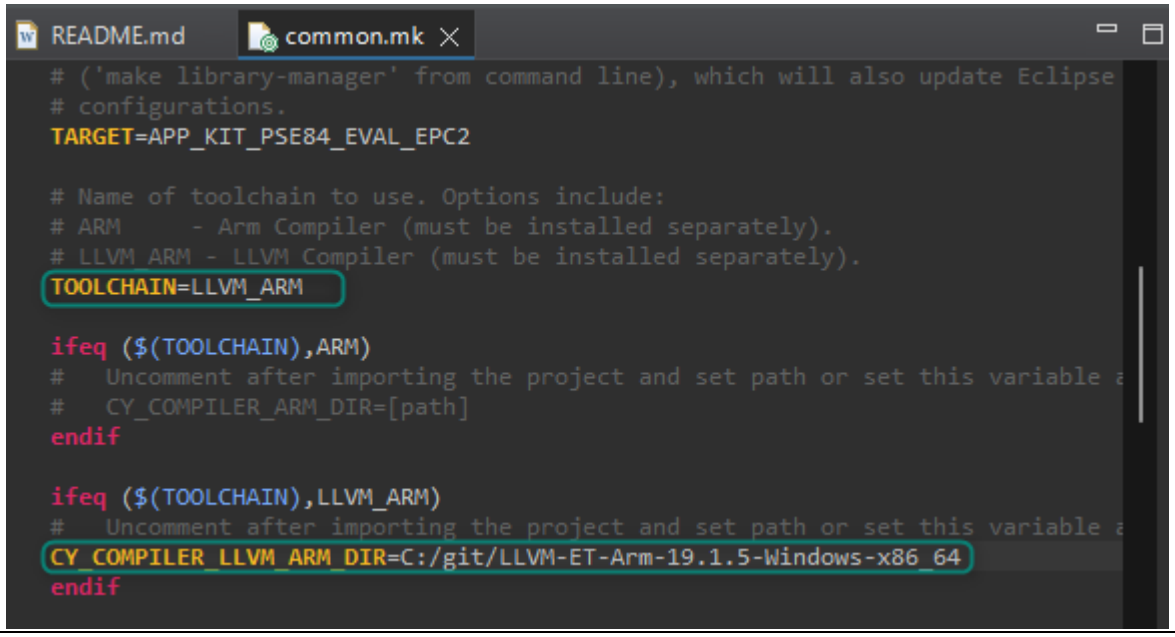


Figure 20 Modifying *common.mk* to use LLVM

4.6 Output

Once running, you should observe the music player on the display. The terminal will display a message and the supported commands.

Note that the BOOT switch must be set to OFF as mentioned in the Hardware overview section.

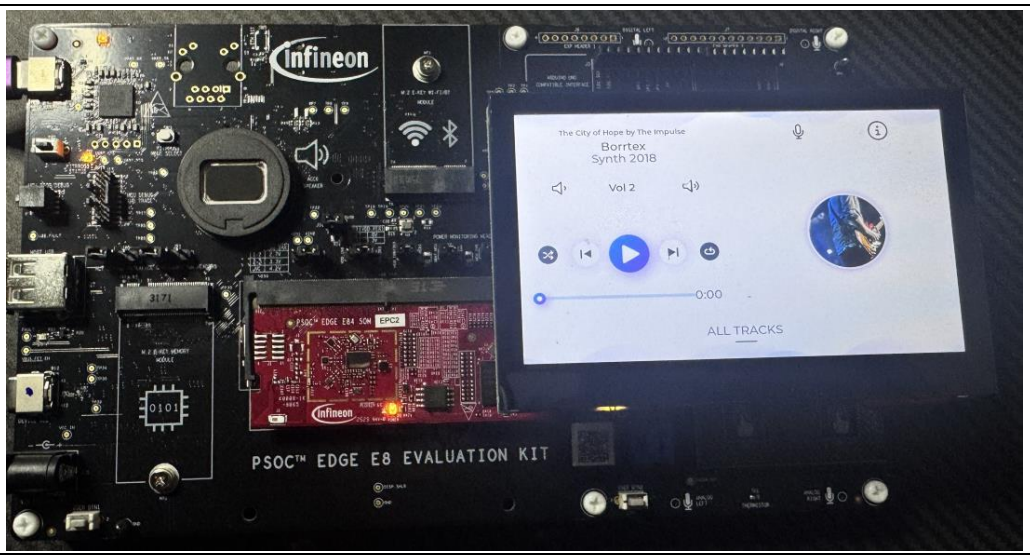


Figure 21 Music player running on EVK

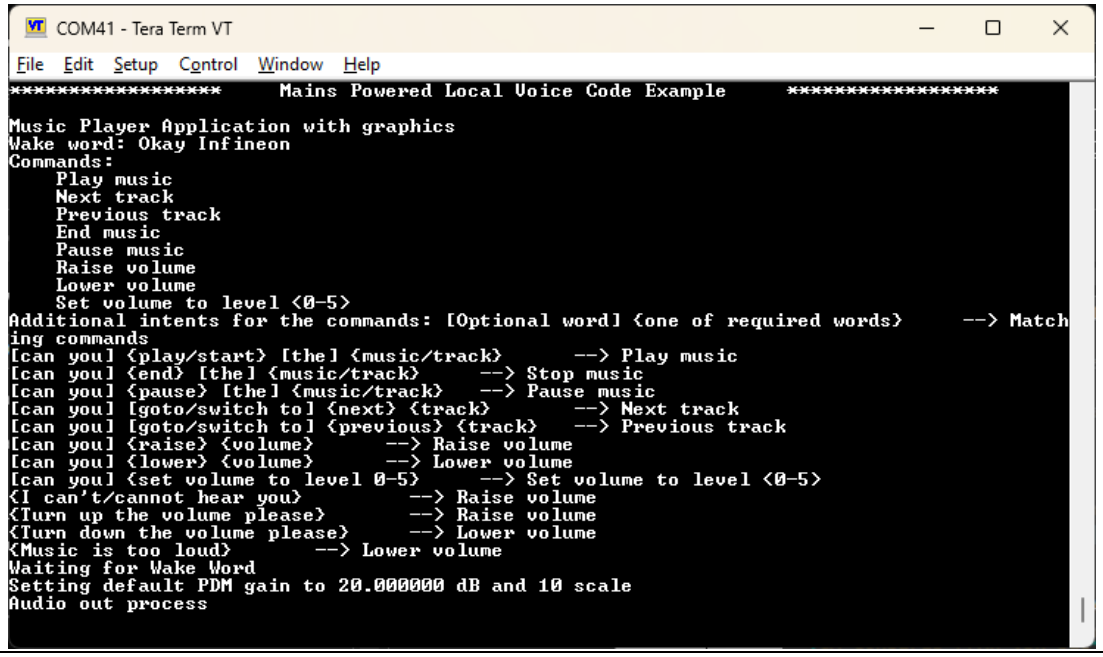


Figure 22 Terminal output of mains powered local voice example

Say the wake word "**OK Infineon**", followed by one of the supported commands, such as "**Play Music**".

Observe the output in the terminal and the application.

4.7 Modify application

We have learned how to use the default mains powered local voice code example using the wake word "OK Infineon". Now, we will learn how to use DEEPCRAFT™ Voice Assistant to change the wake word, and how to add a new command.

Hints for the advanced modification

- Use the DEEPCRAFT™ Voice Assistant to modify the wake word
- A new project can be created, or the voice assistant project included in the code example can be reused
- Update the va_model
- Add any new intent macros and command IDs

Solution

1. Log in to DEEPCRAFT™ Voice Assistant.
2. Click **Import Project**.



Figure 23 Import project in DEEPCRAFT™ Voice Assistant

3. Click **Browse file**, then navigate to the location of the voice assistant project: *proj_cm55\source\mains_powered_application\inferencing_interface\COMPONENT_VOICE_ASSISTANT\va_proj\localvoice_music_nonum.vaproj*
4. Add a **Voice Project Name** and click **Import**.

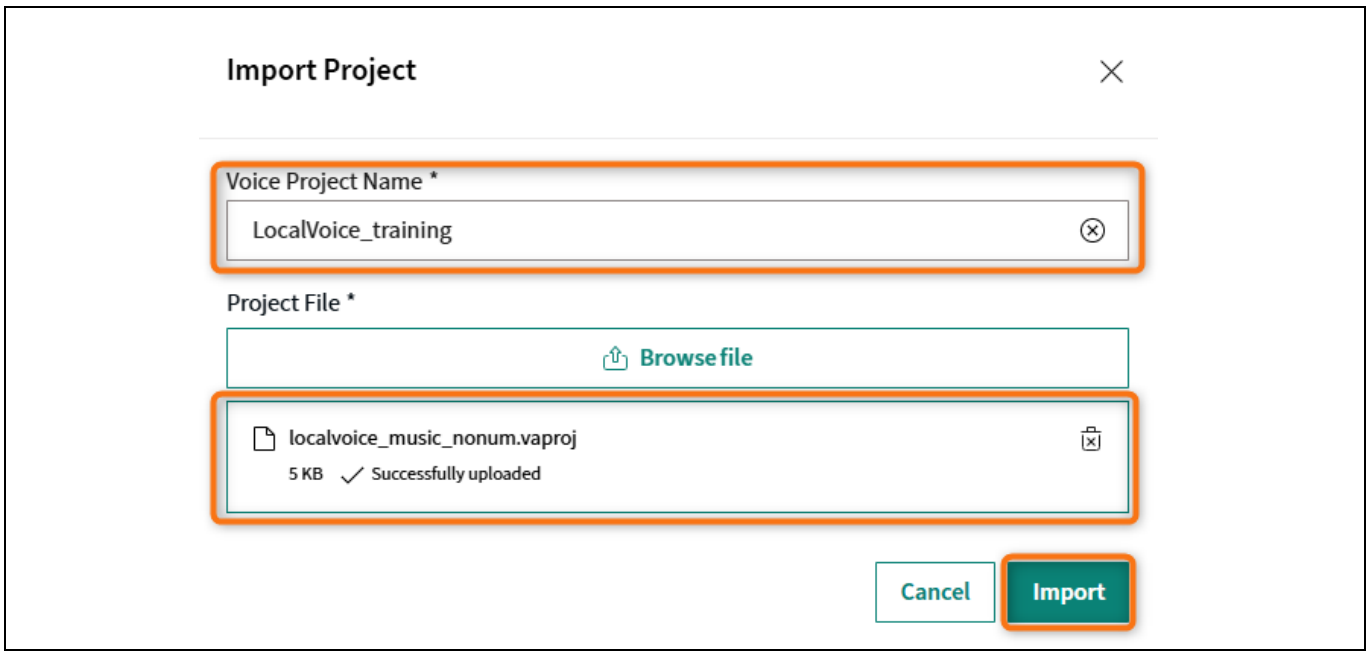


Figure 24 Name VA project and import

5. After the project is imported, click on **Edit**.

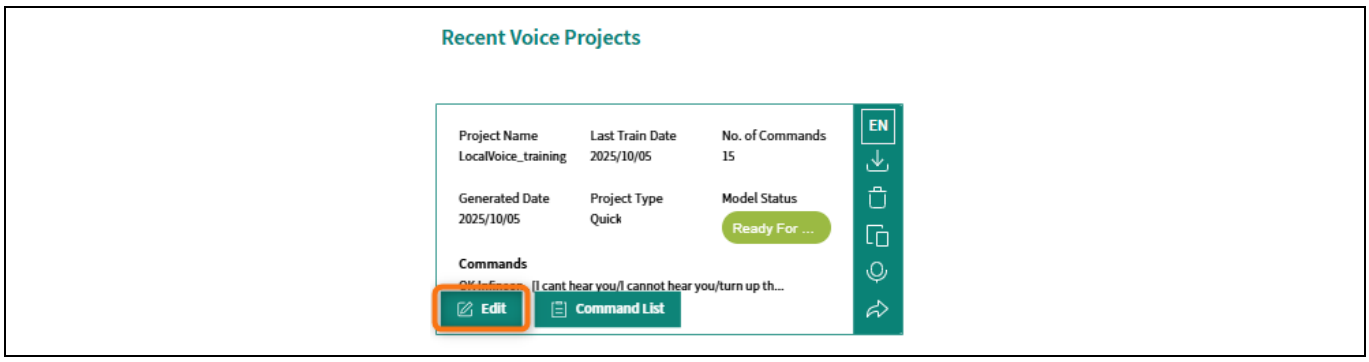


Figure 25 Edit the imported project

6. **Note**: Observe the default structure of the voice assistant project used for this code example. Note that the **wake word** is on the left side in green, and it connects to multiple **text commands** in orange, which trigger individual **intents** in purple.
   Some of the commands are **required** while others are **optional**, so the following commands will trigger the same intent:
   - OK Infineon, [can you] play [the] music → (play music)
   - OK Infineon, [can you] start [the] music → (play music)

- OK Infineon, [can you] play [the] track → (play music)
- OK Infineon, [can you] start [the] track → (play music)

7. Zoom in on the wake word on the left side, click on it to open the **Properties**. Change the **Interjection** to "Hey". Then delete the current **wake word** "Infineon" and replace it with a word of your choice, like "Edge". Press Enter to add the wake word.

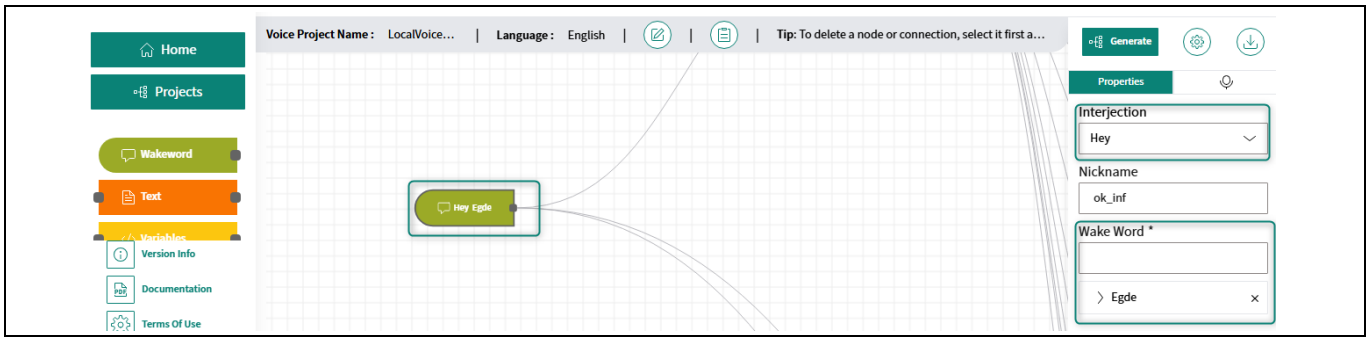


Figure 26 Modify the wake word

8. Navigate to the commands at the top to play music. Add "my" as optional text, and "song" to the list of supported text.
   Note: This will add support for the following command:
   - Hey Edge, [can you] play [my] song→ (play music)

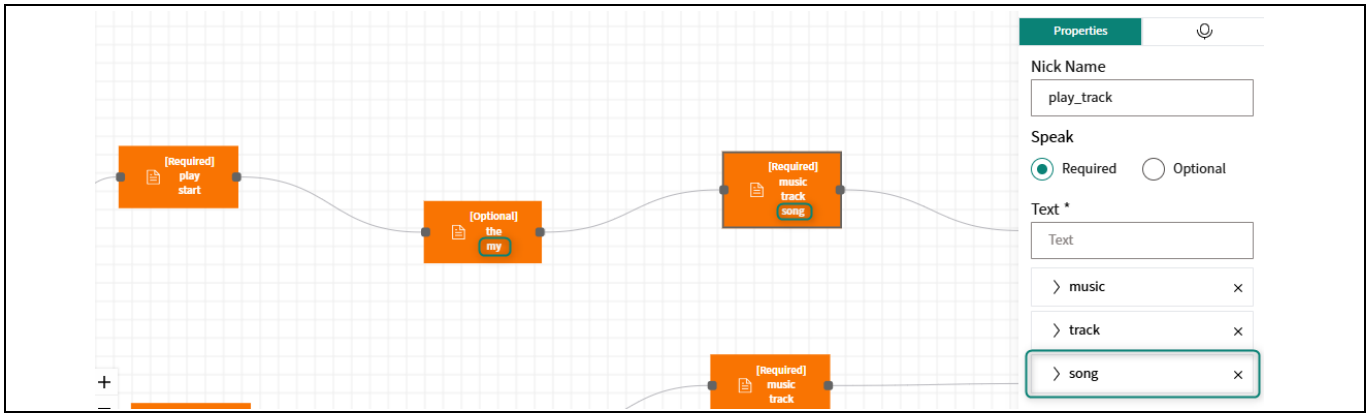


Figure 27 Modify command to play music

9. Navigate to the intent connected to this command and modify the **Intent Name** to "play_song".
   Note: This step is not necessary to change the command, but will be used to explain how to change the intent in the source code.

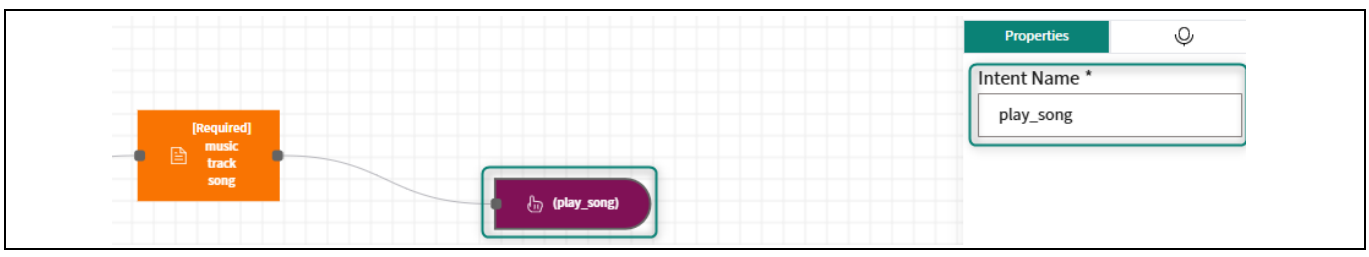


Figure 28 Modify the intent

10. Open Settings to confirm that the project is set for **PSOC™ Edge E84: CM55 + U55** and click on **Generate**.
Note: This step can take several minutes.

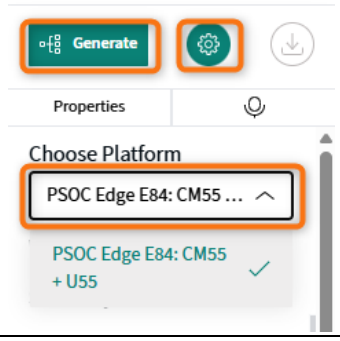


Figure 29 Generate VA model

11. Once the model is generated, download the files.

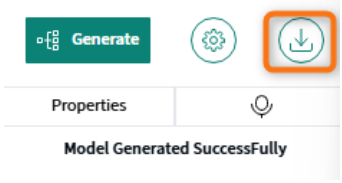


Figure 30 Download VA files

12. Copy-paste all downloaded files to your ModusToolbox™ project in the following location:
<PROJ_DIR>/PSOC_Edge_Mains_Powered_Local_Voice.proj_cm55/source/mains_powered_application/inferencing_interface/COMPONENT_VOICE_ASSISTANT/va_model
Note: Delete previous contents or overwrite any files as needed.

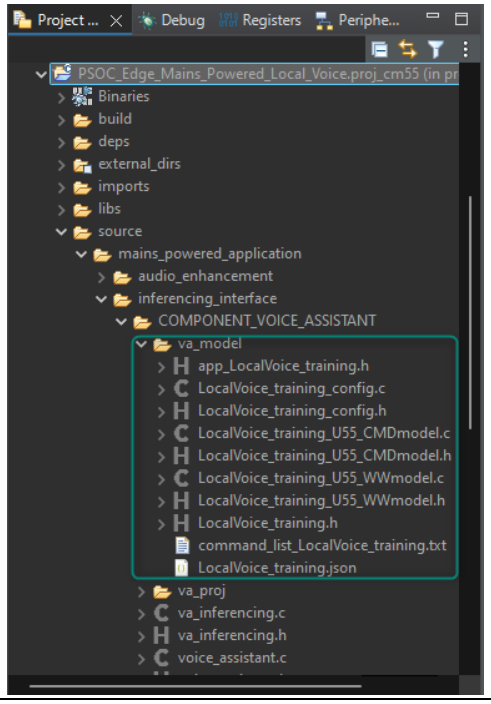


Figure 31 Update VA files in project

13. In *proj_cm55/Makefile*, set **DEEPCRAFT_PROJECT_NAME=[name of your VA project]**, using the VA cloud project name prefixed for the downloaded files.

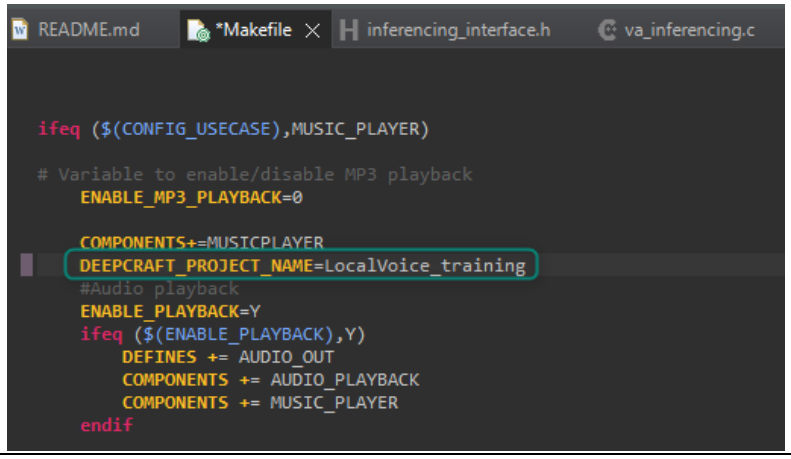


Figure 32 Modify proj_cm55/Makefile

14. We modified the intent name, so we can modify the corresponding macro.
Make the following modifications to
proj_cm55\source\mains_powered_application\inferencing_interface\inferencing_interface.h.

Code listing 2 Modification 1 to *inferencing_inteface.h*

```
typedef enum cy_grp_2_asr_cmds_music
{
    PLAY_SONG_CMD_ID = 401,
    STOP_MUSIC_CMD_ID,
```

Code listing 3 Modification 2 to *inferencing_inteface.h*

```
#define INF_WAKE_WORD                "WAKEWORD"

#define INTENT_PLAY_SONG             "play_song"
#define INTENT_STOP_MUSIC            "end_music"
#define INTENT_INCREASE_VOLUME       "raise_volume"
```

15. **Note**: DEEPCRAFT™ Voice Assistant generates intent macros that can be used in the application; however, in this case, we are only showing how to modify the macros for demonstration purposes. The macros generated by the tool are available in:
proj_cm55\source\mains_powered_application\inferencing_interface\COMPONENT_VOICE_ASSISTANT\va_model\app_[project_name].h
16. Open *proj_cm55\source\mains_powered_application\inferencing_interface\COMPONENT_VOICE_ASSISTANT\va_inferencing.c* and make the modifications shown below. *Observe how the function va_intent_to_id()* sends the command ID to the control task.

Code listing 4 Modification to *va_inferencing.c*

```
void va_intent_to_id (char *intent)
{
    int map_id=0;

    app_log_print("Detected intent %s \r\n",intent);

    if (!strcmp(intent,INF_WAKE_WORD))
    {
        map_id = WAKEWORD_CMD_ID;
    }
    else if (!strcmp(intent,INTENT_PLAY_SONG))
    {
        map_id = PLAY_SONG_CMD_ID;
    }
    else if (!strcmp(intent,INTENT_STOP_MUSIC))
    {
        map_id = STOP_MUSIC_CMD_ID;
    }
```

17. Open *proj_cm55\source\mains_powered_application\control_task.c* and update as shown below. Observe how the code processes the commands and performs different actions.

Code listing 5 Modification to *control_task.c*

```
                case PLAY_SONG_CMD_ID:
                case STOP_MUSIC_CMD_ID:
                case NEXT_TRACK_CMD_ID:
                case PREVIOUS_TRACK_CMD_ID:
                case PAUSE_MUSIC_CMD_ID:
                case INCREASE_VOL_CMD_ID:
                case DECREASE_VOL_CMD_ID:
                case VOL_LEVEL_0_CMD_ID:
                case VOL_LEVEL_1_CMD_ID:
                case VOL_LEVEL_2_CMD_ID:
                case VOL_LEVEL_3_CMD_ID:
                case VOL_LEVEL_4_CMD_ID:
                case VOL_LEVEL_5_CMD_ID:
                {
#ifdef AUDIO_OUT
                    music_player_q_data.data = NULL;
                    music_player_q_data.data_len = 0;
```

```
#endif
                    app_log_print("ASR command detected \r\n");
                    if (map_id==PLAY_SONG_CMD_ID)
                    {
                        app_log_print("Play Song \r\n");
#ifdef AUDIO_OUT
                        i2s_control_flag = I2S_PLAYBACK_PLAY_MUSIC;
                        asr_flag=1;
#endif /* AUDIO_OUT */
                    }
```

18. Lastly, update *proj_cm55\source\use_case\COMPONENT_MUSICPLAYER\COMPONENT_GFX_UI\music_player_gfx_task.c* to handle the new intent in the music player demo.

Code listing 6 Modification to *music_player_gfx_task.c*

```
for (;;)
    {
        /* Check if there's a message in the queue */
        if (xQueueReceive(mp_gfx_queue_handle, &mp_gfx_queue_data, 0) ==
pdTRUE)
        {
            switch(mp_gfx_queue_data.command)
            {
                case OK_INFINEON_CMD_ID:
                {
                    lv_obj_clear_flag(g_spinner, LV_OBJ_FLAG_HIDDEN);
                    lv_obj_add_flag(g_img, LV_OBJ_FLAG_HIDDEN);
                    lv_demo_music_pause();
                    break;
                }
                case PLAY_SONG_CMD_ID:
                {
                    lv_obj_clear_flag(g_img, LV_OBJ_FLAG_HIDDEN);
                    lv_obj_add_flag(g_spinner, LV_OBJ_FLAG_HIDDEN);
                    lv_demo_music_resume();
                    break;
                }
```

19. Rebuild and reprogram the device. Use the new wake word “Hey Edge” followed by the command “[can you] play [my] song” to observe the changes.

5 Appendix A: Creating a PSOC™ Edge application in ModusToolbox™

The following steps show how to create a new project for PSOC™ Edge in ModusToolbox™.

1. Open the **ModusToolbox™ Dashboard 3.6** and launch the **Eclipse IDE for ModusToolbox™**.

Note: If you have not installed ModusToolbox™ or Eclipse IDE, see the Prerequisite section.

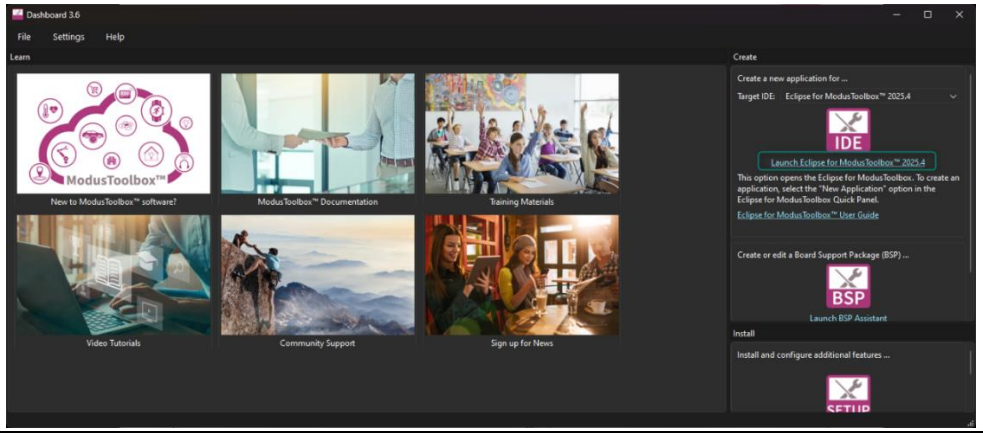


Figure 33 Open Eclipse IDE for ModusToolbox™ from Dashboard

2. Choose a **workspace directory** for your project.
   A folder will be created if the workspace does not exist.
   After selecting a suitable directory, click the **Launch** button.
   Note: It is recommended to keep the workspace path short. Windows has a 260 characters path-length limit and a long workspace path may cause build issues when creating some applications.

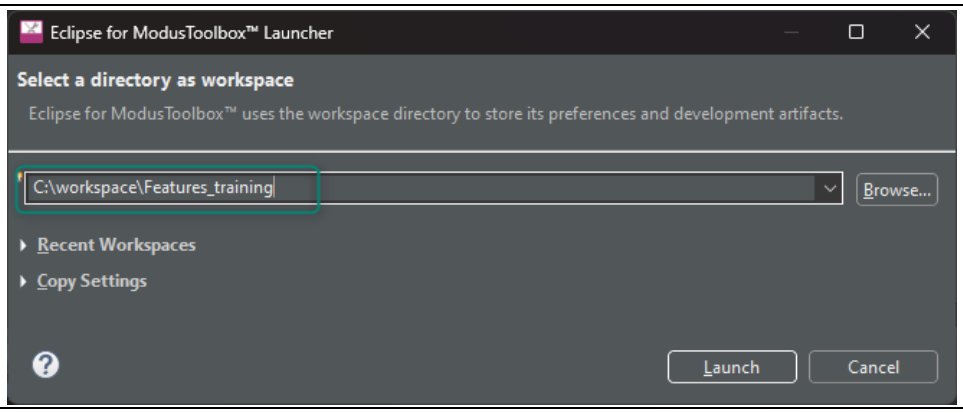


Figure 34 Choose workspace directory

3. Select **New Application** from the Quick Panel. Alternatively, go to **File> New> ModusToolbox™** application. This will launch the **Project Creator Tool**.

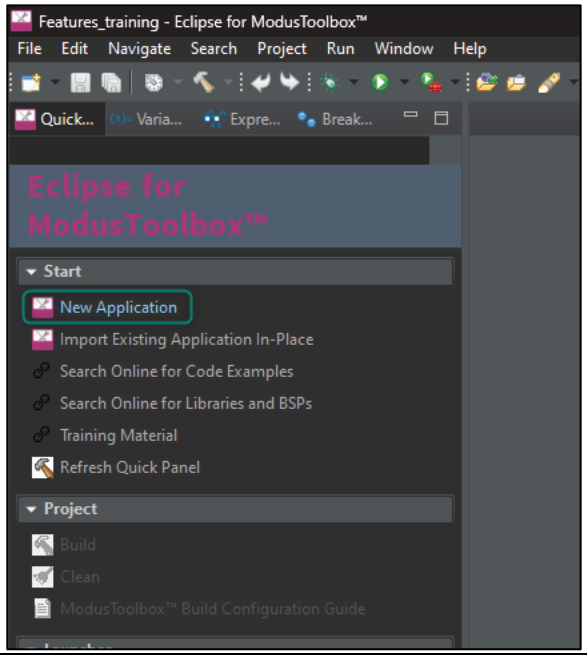


Figure 35 Create new application

4. Once **Projector Creator** launches, you will be prompted to select a board support package (BSP).
   Select the **KIT_PSE84_EVAL_EPC2 BSP** under the PSOC™ Edge BSPs and click **Next**.
   *Note: BSPs are aligned with our development/evaluation kits; they provide files for basic device functionality. A BSP typically has a **design.modus** file that configures clocks and other board-specific capabilities. That file is used by the ModusToolbox™ configurators. A BSP also includes the required device support code for the device on the board. You can modify the configuration to suit your application.*
   Observe that ModusToolbox™ also includes BSPs for PSOC™ Edge EPC4 devices (KIT_PSE84_EVAL_EPC4), and for the PSOC™ Edge AI Kit (KIT_PSE84_AI); however, this training is intended to be used with the KIT_PSE84_EVAL which includes EPC2 devices by default.

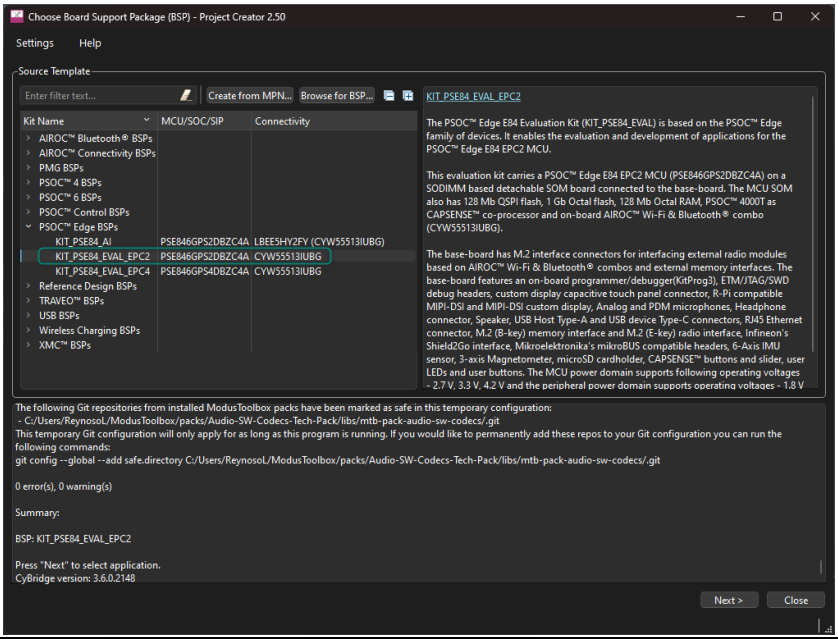


Figure 36 Selecting BSP in Project Creator

5. A new window opens to select an application. The Project Creator includes categories and a search field to make it easier to find and select different examples.

   Click the **checkbox** near the application, optionally rename the project, and then click **Create**.

   Figure 37 shows the creation of the **PSOC™ Edge Hello World** application under the **Getting Started section**.

   Note: Windows has a 260 characters path-length limit. A long workspace path and/or a long project name may cause build issues when creating some applications.

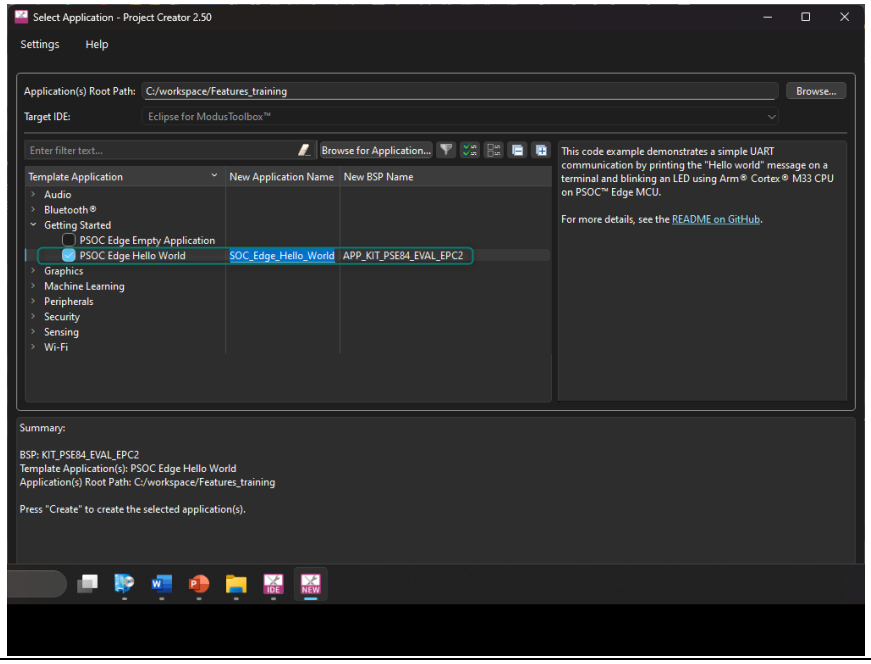


Figure 37 Selecting application

6. A new project is created, and with all the files visible in the **Project Explorer** window.

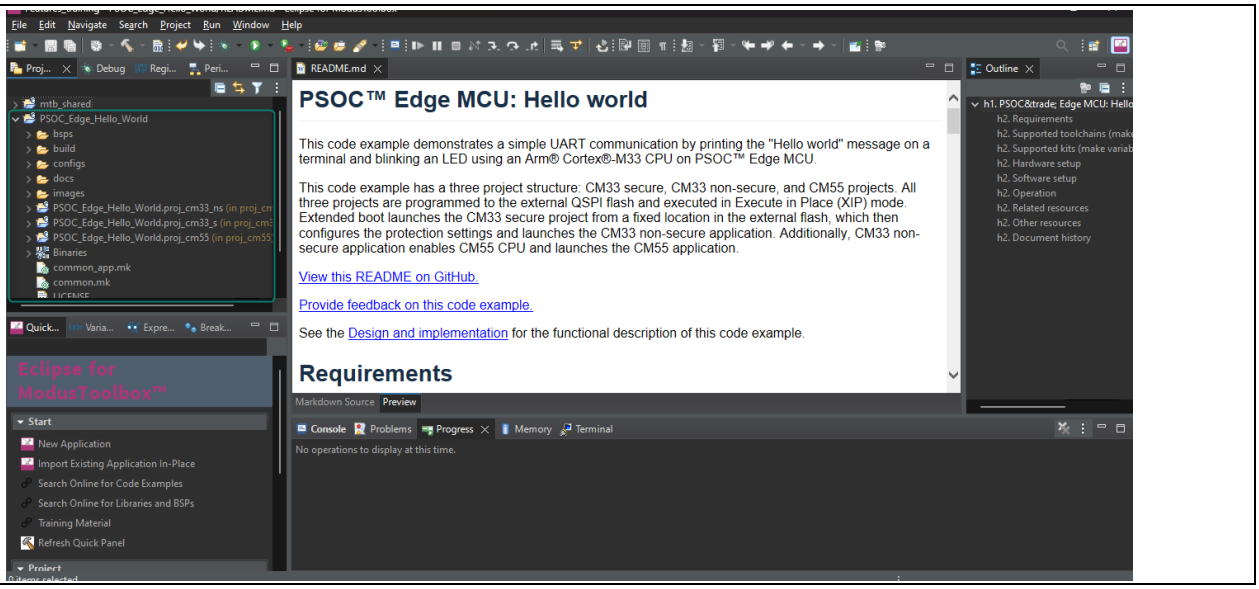


Figure 38 New application created

6 Appendix B: KIT_PSE84_EVAL details

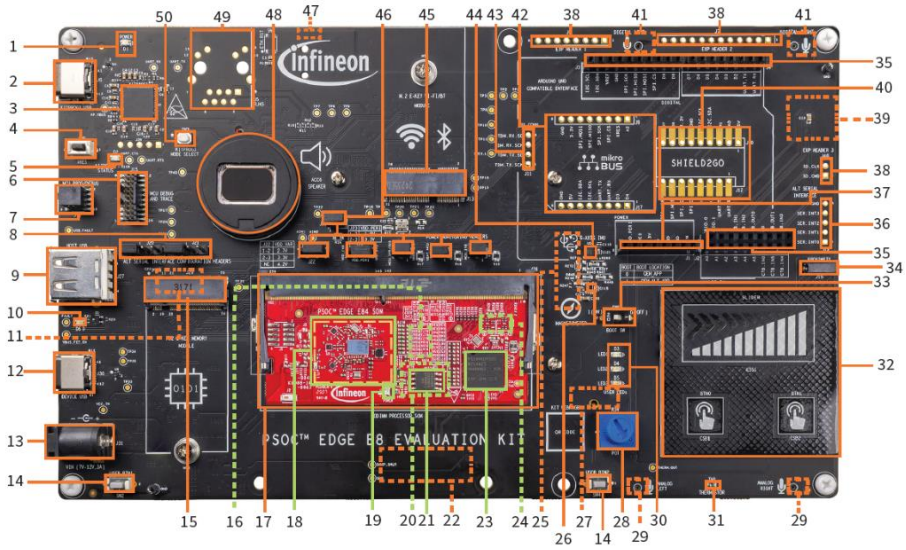


Figure 39 PSOC™ Edge evaluation kit (EVK) details

1 Base board power LED (D1)
2 KitProg3 program/debug USB Type-C connector (J8)
3 PSOC™ 5LP-based KitProg3 programmer and debugger (CY8C5868LTI-LP039, U2)
4 Reset button (SW1)
5 KitProg3 status LED (D2)
6 PSOC™ Edge E84 MCU ETM/JTAG debug and trace header (J15)
7 PSOC™ Edge E84 MCU 10-pin SWD/JTAG program and debug header (J16)
8 Alternative serial interface configuration headers (J20, J21)
9 PSOC™ Edge E84 MCU USB host Type-A connector (J27)
10 USB Type-C Power Delivery (PD) fault LED (D6)
11 Custom display capacitive touch panel connector (J37)**
12 PSOC™ Edge E84 MCU USB device Type-C connector (J30)
13 External power supply VIN connector (J31)
14 PSOC™ Edge E84 MCU user buttons (SW2, SW4)
15 M.2 (B-key) memory interface connector (J29)
16 128-Mb Octal-SPI HYPERRAM™ (S70KS1283GABHI020, U12)***
17 Processor system-on-module (SoM) 260-pin SODIMM connector (J28)
18 AIROC™ CYW55513 tri-band (Wi-Fi & Bluetooth®) combo radio (U3) section
19 Processor system-on-module (SoM) power LED (D3)
20 1-Gb Octal-SPI NOR flash (S28HS01GTGZBH1030, U10)***
21 128-Mb Quad-SPI NOR flash (S25FS128SAGMFB100, U11)
22 MIPI-DSI custom display connector (J38)**
23 PSOC™ Edge E84 MCU (PSE846GPS2DBZC4A, U1)
24 PSOC™ 4000T CAPSENSE™ Co-processor (U9)***
25 Raspberry Pi-compatible MIPI-DSI display connector (J39)**
26 3-axis magnetometer (U4)
27 Raspberry Pi compatible display capacitive touch connector (J41)**
28 Linear potentiometer (R34)
29 Analog microphones (IM73A135V01XTSA1, U36 and U37)**
30 User LEDs (D3, D4, D5)
31 Thermistor (TH1)
32 CAPSENSE™ buttons and slider (CSB1, CSB2, CSS1)
33 BOOT configuration switch (SW6)
34 Proximity sense connector (J19)
35 I/O headers compatible with Arduino UNO R3 (J2, J3, J4)
36 Alternative serial interface I/O header (J14)*
37 Power header compatible with Arduino UNO R3 (J1)
38 PSOC™ Edge E84 MCU expansion I/O headers (J6, J7, J40)*
39 MicroSD card holder (J35)**
40 Infineon Shield2Go interface headers (J10, J12)*
41 PDM microphones (IM72D128V01XTMA1, U7 and U8)**
42 mikroBUS compatible headers by Mikroelektronika (J9, J17)*
43 Extended I2S header (J11)*
44 6-axis accelerometer and gyroscope IMU (U5)
45 M.2 (E-key) radio interface connector (J13)
46 PSOC™ Edge E84 MCU power selection/ monitoring headers (J18, J22, J23, J24, J25, J26)
47 Headphone connector (J34)*
48 Speaker (ACC6)
49 RJ45 ethernet magjack connector (J5)*
50 KitProg3 programming mode selection button (SW3)

* Footprint only, not populated on the board
** Component at the bottom side of the Baseboard
*** Component at the bottom side of the SoM

Revision history

Document revision	Date	Description of changes
**	2026-01-15	Initial release.

Trademarks

All referenced product or service names and trademarks are the property of their respective owners.

PSOC™, formerly known as PSoC™, is a trademark of Infineon Technologies. Any references to PSoC™ in this document or others shall be deemed to refer to PSOC™.

Edition 2026-01-15

Published by

Infineon Technologies AG

81726 Munich, Germany

© 2026 Infineon Technologies AG. All Rights Reserved.

Do you have a question about this document?

Email:

erratum@infineon.com

Important Notice

Products which may also include samples and may be comprised of hardware or software or both ("Product(s)") are sold or provided and delivered by Infineon Technologies AG and its affiliates ("Infineon") subject to the terms and conditions of the frame supply contract or other written agreement(s) executed by a customer and Infineon or, in the absence of the foregoing, the applicable Sales Conditions of Infineon. General terms and conditions of a customer or deviations from applicable Sales Conditions of Infineon shall only be binding for Infineon if and to the extent Infineon has given its express written consent.

For the avoidance of doubt, Infineon disclaims all warranties of non-infringement of third-party rights and implied warranties such as warranties of fitness for a specific use/purpose or merchantability.

Infineon shall not be responsible for any information with respect to samples, the application or customer's specific use of any Product or for any examples or typical values given in this document.

The data contained in this document is exclusively intended for technically qualified and skilled customer representatives. It is the responsibility of the customer to evaluate the suitability of the Product for the intended application and the customer's specific use and to verify all relevant technical data contained in this document in the intended application and the customer's specific use. The customer is responsible for properly designing, programming, and testing the functionality and safety of the intended application, as well as complying with any legal requirements related to its use.

Unless otherwise explicitly approved by Infineon, Products may not be used in any application where a failure of the Products or any consequences of the use thereof can reasonably be expected to result in personal injury. However, the foregoing shall not prevent the customer from using any Product in such fields of use that Infineon has explicitly designed and sold it for, provided that the overall responsibility for the application lies with the customer.

Infineon expressly reserves the right to use its content for commercial text and data mining (TDM) according to applicable laws, e.g. Section 44b of the German Copyright Act (UrhG).If the Product includes security features:

Because no computing device can be absolutely secure, and despite security measures implemented in the Product, Infineon does not guarantee that the Product will be free from intrusion, data theft or loss, or other breaches ("Security Breaches"), and Infineon shall have no liability arising out of any Security Breaches.

If this document includes or references software:

The software is owned by Infineon under the intellectual property laws and treaties of the United States, Germany, and other countries worldwide. All rights reserved. Therefore, you may use the software only as provided in the software license agreement accompanying the software.

If no software license agreement applies, Infineon hereby grants you a personal, non-exclusive, non-transferable license (without the right to sublicense) under its intellectual property rights in the software (a) for software provided in source code form, to modify and reproduce the software solely for use with Infineon hardware products, only internally within your organization, and (b) to distribute the software in binary code form externally to end users, solely for use on Infineon hardware products. Any other use, reproduction, modification, translation, or compilation of the software is prohibited. For further information on the Product, technology, delivery terms and conditions, and prices, please contact your nearest Infineon office or visit https://www.infineon.com